

As with skin states, you can look at the documentation to find out which skin parts are required by the host component. To declare the Application component's required contentGroup skin part in CustomAppSkin.mxml, place the cursor above the closing `</s:Application>` tag and add the following code:

- `<s:Group id="contentGroup"/>`

At this point, the custom skin should look like,

- `<?xml version="1.0" encoding="utf-8"?>`
- `<s:SparkSkin xmlns:fx="http://ns.adobe.com/mxml/2009"`
- `xmlns:s="library://ns.adobe.com/flex/spark"`
- `xmlns:mx="library://ns.adobe.com/flex/mx">`
- `<fx:Metadata>`
- `[HostComponent("spark.components.Application")]`
- `</fx:Metadata>`
- `<fx:Declarations>`
- `<!-- Place non-visual elements`
- `(e.g., services, value objects) here -->`
- `</fx:Declarations>`
- `<s:states>`
- `<s:State name="normal" />`
- `<s:State name="disabled" />`
- `</s:states>`
- `<s:Group id="contentGroup"/>`
- `</s:SparkSkin>`

The custom skin doesn't implement a visual design yet, but it has everything you need to make it compatible with the Spark Application component. The next step is to add visual objects to the skin. Because the custom skin uses the default basic layout, objects can be overlaid on top of each other, and you can use constraint-based layout properties such as top, left, bottom, and right to position objects relative to the skin's borders.

Visual objects that are added to the layout in the host component should appear in the application's background. To make this happen, just declare any FXG graphics in the skin component's code above the declaration of the contentGroup object.

To add a background color and a frame with a drop shadow to the application's appearance, in CustomAppSkin.mxml, place the cursor above the Group with an id of contentGroup and make a new blank line. Then declare a rectangle that covers the entire application canvas and has a solid light gray color of #CCCCCC: